Trace-Level Failure Diagnostics for Agentic AI: A Cross-Benchmark Audit with MAS Analyzer

Anonymous Authors[1]

Abstract

Terminal success rates hide many of the failures that matter in long-horizon agentic systems. A run may answer with the wrong hypothesis after weak retrieval, declare impossibility too early, or continue coordinating after the underlying tool state has already stalled. We present a trace-centered audit of a multi-agent evaluation suite produced with MAS Analyzer across five benchmarks and seven topologies. The corpus contains 1,050 runs in total; for the best topology of each benchmark, we audit all 330 failed runs using benchmark summaries, trajectory artifacts, and event-level traces. The audit yields operational definitions for recurrent failure modes with explicit trace boundaries: browse-style tasks are dominated by evidence insufficiency, planning tasks by premature impossibility, tool-heavy tasks by interface fragility, workplace tasks by grounding and workflow blocks, and financial tasks by support fragility. Our main conclusion is that trace diagnostics should be treated as a first-class output of agent evaluation rather than as post-hoc anecdotal analysis.

1. Introduction

Agentic AI systems fail in ways that final accuracy alone cannot explain. A bad assumption at the beginning of a run may silently contaminate later decisions; a retrieval loop may remain active without learning anything new; or a workflow may be operationally correct in the abstract but blocked because the agent cannot ground the necessary person, document, or tool response. These errors are especially important in long-horizon settings because they tend to accumulate before they become visible in the final answer.

This paper studies such failures through traces rather than through terminal scores alone. We analyze a MAS Analyzer experiment suite spanning five benchmarks—browsecomp, plancraft, stabletoolbench, workbench, and finance_agent—evaluated under seven topologies. The suite contains 30 tasks per benchmark with three repeated runs per task. The resulting artifacts include benchmark-level summaries, per-run trajectories, trace metrics, and event-level JSONL logs. Together, these artifacts allow us to ask not only which topology performs best, but also what kind of failure appears when a run loses.

Our central claim is that agentic failure is structured. The dominant error patterns differ by benchmark, and coordination topology changes the cost profile even when it does not change the type of failure. We frame these patterns as falsifiable mechanistic hypotheses about how long-horizon drift emerges: weak evidence can lock a run into a wrong hypothesis, local scarcity can be misread as global impossibility, tool fragility can propagate upward into synthesis errors, and unresolved entities can block otherwise correct workflows. To support that claim, we pair aggregate outcome tables with a failed-run audit over the best-performing topology for each benchmark. This framing turns the paper from a leaderboard report into a diagnostic study.

2. Related Work

Our study sits at the intersection of agent architectures, tool use, and process-level evaluation. ReAct couples reasoning with environment interaction and remains a foundational reference for trace-aware agent behavior (Yao et al., 2023). Toolformer studies self-supervised tool invocation (Schick et al., 2023), Reflexion emphasizes iterative verbal improvement after failure (Shinn et al., 2023), and AutoGen popularized multi-agent conversation as a systems pattern (Wu

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country.AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

et al., 2023). These works show that reasoning, action, memory, and coordination all matter; they do not, however, by themselves explain why a full run fails in a particular benchmark.

Benchmark work has made the need for process-aware evaluation increasingly clear. AgentBench highlighted failure modes in long-horizon agent execution (Liu et al., 2023), while WebArena demonstrated how realistic environments expose brittle online behavior even for strong models (Zhou et al., 2023). Our contribution is complementary. Rather than proposing a new benchmark or a new topology, we use a common artifact format to compare failure structure across benchmarks. The resulting output is closer to a trace audit and failure taxonomy than to a new performance leaderboard.

This difference matters because most benchmark reporting still privileges issue-driven trends in terminal correctness: whether the answer string is right, whether the task was marked complete, or whether a tool call succeeded at least once. Our audit suggests that many long-horizon failures are interface-level and state-dependent rather than purely model-internal. In several of the traces studied here, the decisive event is not a single bad thought, but a mismatch between the current evidence state and the confidence with which the system proceeds. This is why our analysis is intended to complement, rather than replace, standard benchmark reporting.

Our findings also qualify some expectations created by self-reflection and tool-use papers. ReAct-style interleaving can keep an agent grounded only when the environment returns support that is actually decision-relevant. Reflexive re-planning can help only when the agent recognizes the right failure boundary. In finance-style tasks with strict evidence requirements, or browse-style tasks with weak retrieval support, reflection often arrives after the support chain has already degraded. Likewise, multi-agent discussion does not automatically yield adversarial robustness. When every agent sees the same thin evidence slice, discussion can become redundant confirmation rather than corrective scrutiny. This paper therefore enters into dialogue with prior work by asking not only whether agents can act, reflect, or coordinate, but under what trace conditions those mechanisms fail to arrest error propagation.

3. Study Design

3.1. Experimental Corpus

We analyze one MAS Analyzer experiment suite rooted in an anonymized artifact bundle. The suite evaluates seven topologies:

- sas
- only_voting
- fully_linked_debate
- group_chat_debate
- orchestrator_no_discussion
- orchestrator_tree_structure
- orchestrator_with_discussion

Each benchmark contributes 30 tasks and each task is run three times, yielding 210 runs per benchmark and 1,050 runs in total. The experiment summary records benchmark-level success, score, and stability; topology-level descriptors record aggregate token and communication metrics; and each task directory stores per-run trajectories, evaluation outputs, and traces.

3.2. Failure Audit Protocol

The paper uses a two-stage analysis. First, we report aggregate benchmark outcomes across all seven topologies. Second, for each benchmark we select the best-performing topology by task success rate and audit every failed run under that topology. This produces an audit set of 330 failed runs: 80 for browsecomp, 56 for plancraft, 39 for stabletoolbench, 71 for workbench, and 84 for finance_agent.

Each failed run receives one primary failure label. We seed the audit with phrase-level cues from the final answer and final reason, then inspect representative trajectories and traces for each proposed class. The goal is not to claim a universal ontology. Instead, we seek the smallest set of benchmark-grounded labels that explains the dominant failure shapes visible in this corpus. When a run exhibits multiple issues, we assign the label that best explains why the run failed to complete the benchmark objective.

3.3. Labeling Workflow

The labeling workflow deliberately mirrors how a practitioner would debug a failed run while adding enough structure to make the paper auditable. For each benchmark, we first identify the best-performing topology

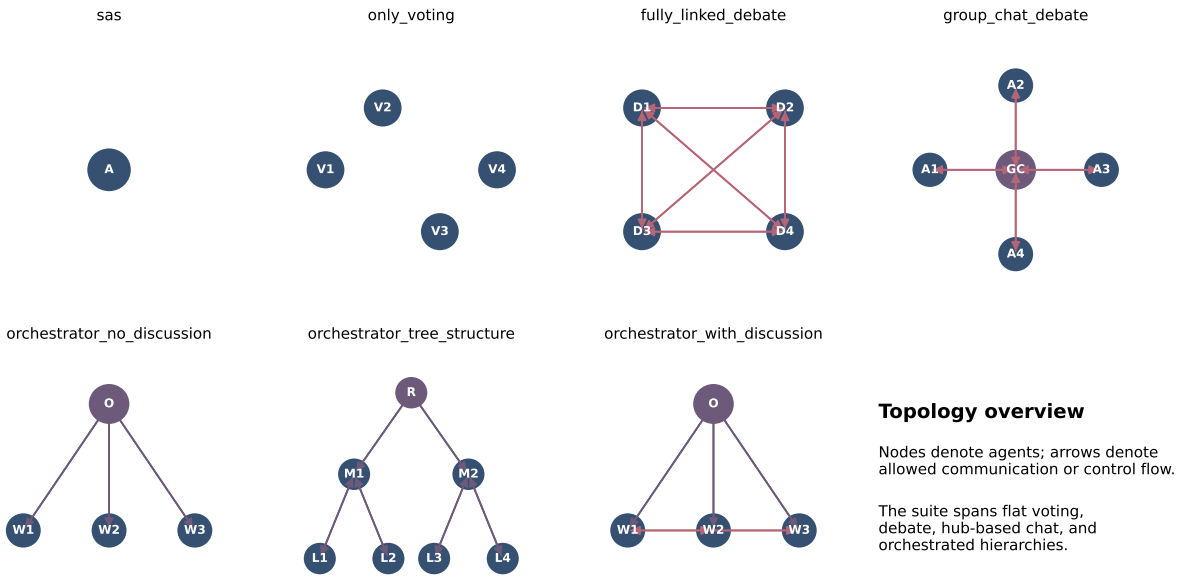

Figure 1. Topology overview for the seven systems studied in this paper. The suite spans single-agent execution, flat voting, fully connected debate, hub-based group chat, and two orchestrated variants with different communication constraints.

from the benchmark summary table. We then enumerate all failed runs under that topology using the per-run evaluation JSON files rather than the final answer string alone. This distinction matters because several benchmarks contain runs that return fluent answers but still receive a failing evaluation.

Each failed run is read in the same order: task prompt, stage contract, visible packets, tool outputs, final reason, final answer, and evaluation result. Phrase-level cues from the final answer provide only an initial routing heuristic. The primary label is assigned only after checking that the surrounding trajectory supports the same explanation. For example, a run that ends with an unsupported numerical answer is not treated as a retrieval failure unless the trace shows that the support chain was actually missing. Likewise, a blocked workflow is coded as a grounding failure only when the missing entity binding is what stops execution.

This workflow has two practical consequences. First, the categories are intentionally benchmark-grounded rather than benchmark-agnostic. Second, the labels are asymmetric: a category such as premature impossibility is narrow because the benchmark grammar makes it easy to diagnose, whereas workflow blocking is broader because workbench failures often combine missing entities, missing sender identity, and missing action affordances in the same run.

3.4. Primary-Label Rule

Some failed runs exhibit multiple issues at once, so we use a single primary-label rule to keep the audit readable. The primary label is the earliest trace-visible condition that best explains why the benchmark objective became unattainable under the observed run. If weak tool evidence leads to a wrong final synthesis, the run is labeled as tool fragility when the tool failure is the better explanation, and as unsupported synthesis when the tool calls are technically successful but the evidence remains too thin to support the final claim. If an entity lookup fails and later causes calendar or CRM actions to stall, the run is labeled as grounding failure rather than generic workflow blocking.

3.5. Research Questions

We organize the paper around four questions.

RQ1: Can trace-level inspection recover stable failure modes? We ask whether the same failure shapes recur often enough to support a compact taxonomy rather than a list of anecdotes.

RQ2: Which benchmarks map to which failure regimes? We ask whether different task families systematically induce different dominant failure patterns.

RQ3: How does topology interact with failure structure? We ask whether coordination changes the kind

Benchmark	Best topology	Success	Tokens	Stability
browsecomp	only_voting	0.111	45.9k	0.911
plancraft	only_voting	0.378	35.1k	0.644
stabletoolbench	group_chat_debate	0.567	129.5k	0.437
workbench	only_voting	0.211	71.0k	0.881
finance_agent	orchestrator_tree_structure	0.067	214.8k	1.000

Table 1. Best topology by benchmark in the analyzed run. The winner is benchmark-dependent, and higher success frequently co-occurs with higher token cost.

Benchmark	Best vs. SAS	Best success	SAS success	Token ratio
browsecomp	+0.033	0.111	0.078	4.3×
plancraft	+0.011	0.378	0.367	3.9×
stabletoolbench	+0.233	0.567	0.333	9.7×
workbench	+0.044	0.211	0.167	3.5×
finance_agent	+0.022	0.067	0.044	8.4×

Table 2. Best-system gains relative to the single-agent baseline. StableToolBench benefits the most from coordination, but also at the largest token overhead.

of failure, the cost of reaching it, or both.

RQ4: What trace-local conditions are reproducible triggers? We ask whether the artifacts expose compact trigger patterns that another researcher could use to recreate and stress test the same failure regime.

4. Aggregate Benchmark Outcomes

Tables 1 and 2 already show that there is no universal winner. Three benchmarks prefer only_voting, one prefers group_chat_debate, and one prefers orchestrator_tree_structure. The gains over sas are also highly uneven. On plancraft, the best system is only marginally better than the single-agent baseline. On stabletoolbench, the gain is substantial but the cost is almost ten times larger. This pattern motivates a failure audit rather than a winner-takes-all interpretation: the role of coordination depends on what the benchmark is asking the system to survive.

5. Failed-Run Audit

Tables 3 and 4 and Figure 2 together form the core quantitative result of the paper. They show that the dominant failure mode is benchmark-specific, not topology-agnostic. The taxonomy is also more nuanced than a single "the model was wrong" label.

For browsecomp, most failed runs do not terminate with an explicit blocked status. Instead, they produce a candidate answer that is weakly supported or supported by the wrong chain of evidence. That is why the dominant label is evidence insufficiency / unsupported hypothesis rather than only blocked retrieval. For plancraft, by contrast, every failed run in the best topology ends in an impossible action, making premature impossibility the cleanest failure signature in the entire suite.

Benchmark	Successes in best topology	Failed runs audited
browsecomp	10	80
plancraft	34	56
stabletoolbench	51	39
workbench	19	71
finance_agent	6	84

Table 3. Outcome counts for the best topology of each benchmark. The audit covers every failed run in these best-topology slices.

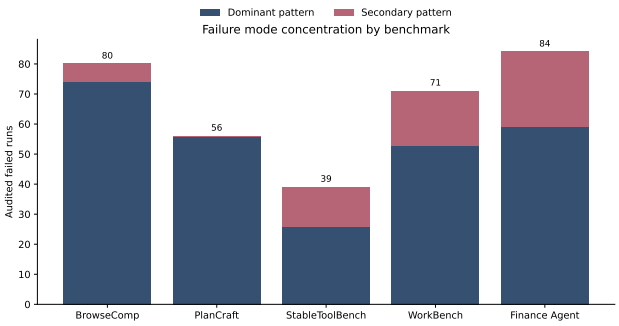

Figure 2. Stacked view of dominant and secondary coded failure patterns. The visual concentration differs sharply by benchmark, with plancraft being the cleanest and finance-agent the most diffuse.

For stabletoolbench, roughly two thirds of failures explicitly cite tool or API limitations, inconsistent outputs, or missing information. The remaining third usually look like synthesis failures after weak evidence rather than clean tool exceptions. For workbench, the failure surface splits between entity-grounding errors and more general workflow blocks. Finally, finance_agent is the hardest benchmark in absolute terms, and its failures are dominated by unsupported financial claims or brittle evidence chains rather than simple retrieval stalls.

5.1. Within-Benchmark Concentration

The audit is also informative about concentration. Plancraft is the most concentrated benchmark: every audited failure in the best topology belongs to the same class. Stabletoolbench is moderately concentrated, with a clear dominant class but a meaningful second class. Browsecomp appears concentrated numerically, but the concentration hides an important distinction between explicit blocked retrieval and fluent unsupported answers. Workbench and finance_agent are the least tidy: both show a dominant class, but both also exhibit enough secondary structure that a single coarse label would obscure the operational

Benchmark	Failed runs audited	Dominant coded pattern	Secondary coded pattern
browsecomp	80	Evidence insufficiency / unsupported hypothesis: 74 (92.5%)	Explicit evidence starvation: 6 (7.5%)
plancraft	56	Premature impossibility: 56 (100.0%)	None observed at comparable frequency
stabletoolbench	39	Tool/interface fragility: 26 (66.7%)	Unsupported synthesis after weak tool evidence: 13 (33.3%)
workbench	71	Workflow blocking: 53 (74.6%)	Entity grounding failure: 18 (25.4%)
finance_agent	84	Financial support gap: 59 (70.2%)	Stack brittleness: 25 (29.8%)

Table 4. Lightweight failed-run audit over the best-performing topology for each benchmark. Percentages are computed within benchmark. The audit is based on all failed runs in the selected topology and is intended as a diagnostic summary rather than a universal ontology.

story.

5.2. Failure Modes Beyond the Winning Topology

Although the paper's audit centers on the best topology for each benchmark, the broader experiment summary suggests that these failure modes are not artifacts of a single system design. Browsecomp remains hard across all topologies. Plancraft stays relatively strong in aggregate even while premature impossibility persists in weaker systems. Stabletoolbench benefits most from discussion, but still carries a large residual class of tool-induced failures. Workbench remains low in absolute success regardless of topology, which is consistent with an environment where missing entity bindings frequently dominate the run. This broader pattern is one reason we interpret the audit as benchmark-specific structure rather than as a quirk of one topology.

6. Benchmark-Specific Failure Modes

6.1. BrowseComp: Evidence Insufficiency and Wrong-Hypothesis Lock-In

Browse-style failures often look active while remaining epistemically stagnant. The system searches repeatedly, obtains partial matches, and then narrows around a candidate before it has assembled a sufficient proof. In the clearest blocked cases, the final answer is an explicit admission such as "blocked" or "the necessary evidence …is not recovered." More commonly, however, the run returns a concrete answer that remains under-supported. In the audit, this pattern accounts for 74 of 80 failures in the best browsecomp topology.

The important observation is that these runs are not purely retrieval failures. The trace usually shows enough surface activity to create the impression of progress, but the evidence state does not improve in a cumulative way. A useful diagnostic question is therefore not "did the agent search?" but "did the search materially change the support for the current hypothesis?"

6.2. Plancraft: Premature Impossibility

Plancraft is the cleanest benchmark in the taxonomy because the failure grammar is explicit. Failed runs overwhelmingly end in an impossible action. What the trajectories reveal, however, is that many of these impossibility judgments are local rather than global. The agent observes a missing prerequisite, fails to explore alternative ingredient routes or transformations, and collapses that local shortage into a global impossibility claim.

This distinction matters. "Locally unavailable now" is not the same as "globally impossible under all valid action sequences." In the audit, all 56 failed runs in the best topology belonged to this class. The benchmark is therefore less about general planning weakness than about a systematic tendency to terminate search too early.

6.3. StableToolBench: Tool Fragility and Unsupported Synthesis

StableToolBench is where coordination helps the most in aggregate, but it is also where the traces most clearly reveal system-level fragility. Many failed runs cite empty API outputs, inconsistent fields, insufficient medical or geographic data, or other interface-level problems. In the audit, these tool/interface issues account for 26 of 39 failures in the best topology.

The remaining 13 failures are also trace-revealing: the agents continue to talk, compare partial outputs, and attempt to synthesize a final answer after the tool evidence has already become too weak to support one. This secondary class is important because it shows that

more discussion does not necessarily repair upstream uncertainty. Sometimes it only converts weak evidence into a more verbose but still unsupported conclusion.

6.4. Workbench: Grounding and Workflow Blocking

Workbench failures are operational. The system may understand what it wants to do, but still fail because it cannot bind the plan to a person, email address, sender identity, or assignee. In the clearest cases, the final answer explicitly reports that no email could be found or that the assignee is unresolved. In other cases, the run is blocked by a more general workflow precondition, such as not being able to retrieve the sender from a prior event.

This is why we separate entity grounding failure from the larger class of workflow blocking. Both matter for real deployment, but grounding is the more benchmark-specific diagnostic signal. The broader lesson is that many enterprise-style failures are not failures of abstract planning at all. They are failures of attachment between action schemas and real-world entities.

6.5. Finance Agent: Support Fragility and Stack Brittleness

Finance-agent failures look different again. The benchmark is extremely hard in absolute terms, and even the best topology succeeds on only 6 of 90 runs. Most failures do not end with clean "blocked" language. Instead, they produce detailed financial answers that are not sufficiently supported by the available evidence chain. We label this dominant pattern financial support gap; it covers 59 of the 84 failures in the best topology.

The remaining 25 failures are better described as stack brittleness. These runs expose the whole system's fragility: tool limitations, inconsistent support documents, and benchmark-specific evidence requirements combine into a failure that is neither pure reasoning error nor simple retrieval stagnation. This benchmark therefore deserves separate treatment rather than being folded uncritically into the same category as web retrieval or workplace execution.

7. Representative Trace Evidence

This section makes the taxonomy more concrete by summarizing representative failures that repeatedly informed the labels.

7.1. BrowseComp Example

One browse-style failure ends with the answer "the necessary evidence to identify the learning institution is not recovered in the current tool responses." The trajectory shows four specialized roles, multiple local-corpus search attempts, and a final majority vote that never transitions from query refinement to sufficient support. This is a useful canonical case because the run is active and structurally coherent, yet the evidence state never crosses the threshold from partial match to defensible answer.

7.2. Plancraft Example

A representative plancraft failure ends with "impossible: Only 1 poppy available for red dye (2 required)." The benchmark grammar makes the failure legible, but the trace reveals the more interesting point: the agent has compressed a local shortage into a global impossibility claim. The run does not fail because the model cannot parse the inventory. It fails because it terminates the search space too early.

7.3. StableToolBench Example

One stabletoolbench failure reports that the available API tools cannot retrieve valid guideline or data fields needed for the answer. Another cites inconsistent property-location evidence from the tool layer. These traces show why we separate tool fragility from unsupported synthesis: in some runs the interface itself is the bottleneck, while in others the tools return something, but not enough to justify the final conclusion.

7.4. Workbench Example

A representative workbench trajectory concludes that a meeting cannot be scheduled because no email can be found for the target person or assignee. The plan itself is not nonsensical. The run reaches the correct operational boundary and then stalls because the final entity binding never materializes. This is exactly the kind of failure that would disappear if we looked only at terminal labels and ignored trajectory structure.

7.5. Finance-Agent Example

Finance-agent failures are often superficially impressive: they contain precise percentages, counts, or growth statements. The problem is not lack of fluency. The problem is that the support chain is brittle, incomplete, or drawn from incompatible evidence sources. This makes finance-agent traces particularly important for separating "numerically specific" from "well supported."

8. Clinical Trace Anatomy

The preceding examples summarize recurring failure signatures. We now deepen two cases in a more clinical style to show how a locally plausible run can drift into a globally unrecoverable state.

8.1. Case Study A: BrowseComp and Unsupported Hypothesis Lock-In

Background. The task requires the system to identify the correct institution associated with a person by assembling a support chain from locally retrieved evidence. Success depends less on isolated keyword matching than on whether multiple retrieved fragments can be linked into a defensible attribution.

Trace timeline. Turn 1: the agents receive the task prompt together with a local retrieval tool and decompose the search into sub-roles around person identity, institution names, and candidate supporting snippets. The initial context is reasonable and the first retrievals return partially overlapping fragments. Turn 2: one agent proposes a candidate institution because its name appears near the target entity in a retrieved document. Another agent notes weak corroboration, but the group still treats the candidate as the working hypothesis and reformulates queries around that institution rather than broadening the search space. Turn 3: subsequent retrievals produce variations of the same evidence neighborhood. The surface text changes, but the evidence state does not materially improve: no new source class is introduced, no decisive relation is verified, and no disconfirming branch is explored in depth. Turn 4: the final vote aggregates these partial observations into a blocked or weakly supported answer. The output is coherent, but coherence arrives without an adequate support chain.

The fatal pivot. The decisive moment is Turn 2, when the candidate institution is upgraded from a provisional lead into the organizing hypothesis for later retrieval. From that point onward, the search loop becomes self-conditioning: each new query is optimized to refine the favored answer instead of to test whether the answer is justified.

Diagnostic analysis. Multi-agent discussion does not repair the run because the agents are debating over nearly the same evidence slice. The missing ingredient is not more conversational bandwidth but a System-2-style obligation to test alternative hypotheses and measure evidence-state change. Once the shared context is already biased toward one under-supported candidate, extra discussion mostly amplifies local confidence. This is a clear example of long-horizon drift: the system remains active, yet each additional turn increases commitment more than epistemic coverage.

8.2. Case Study B: PlanCraft and Premature Impossibility

Background. The task requires constructing a valid crafting plan under inventory and recipe constraints. A failed run does not merely miss a recipe; it incorrectly concludes that no valid sequence exists.

Trace timeline. Turn 1: the agent parses the inventory and target item correctly, identifies a needed intermediate resource, and begins tracing a possible production chain. The early state is promising because the required ingredients are partially present. Turn 2: the planner encounters a local shortage, such as an insufficient count of a dye ingredient. At this point the relevant question should be whether alternate acquisition or transformation routes remain available. Turn 3: instead of expanding those routes, the run collapses the local shortage into a terminal statement of impossibility. The trace contains little evidence of frontier expansion, counterfactual recipe search, or systematic elimination of alternatives. Turn 4: the final action is emitted as impossible, and the evaluation marks the run as failed even though the internal reasoning chain appears locally tidy.

The fatal pivot. The decisive error occurs at Turn 3, where the planner changes the semantics of the state from "current branch blocked" to "goal globally unattainable." That inferential jump is the mechanism of failure.

Diagnostic analysis. Here the problem is not noisy evidence but inadequate search semantics. Even if several agents were to discuss the same state, they would not correct the outcome unless one of them explicitly preserved alternative branches and challenged the global impossibility claim. In this sense, coordination is orthogonal to the bottleneck. The missing capability is logical stability over the planning state: the system needs a formal criterion for when impossibility is licensed, not simply more turns of fluent reasoning.

9. Topology Interaction and Cost

The topology story is more subtle than "more agents help." The aggregate results show that some benchmarks prefer simple aggregation while others benefit from richer discussion. The trace audit explains why.

On browsecomp, extra discussion often amplifies

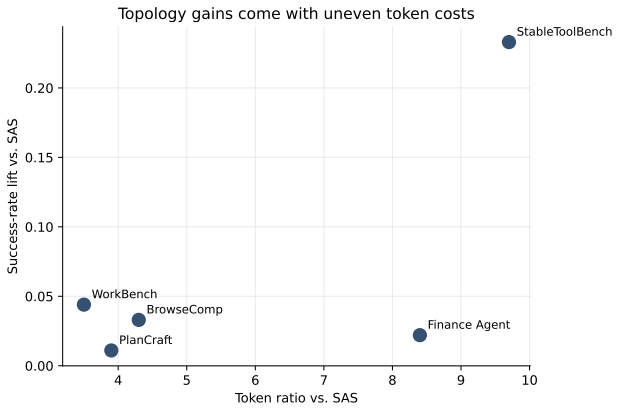

Figure 3. Success-rate lift versus token multiplier relative to sas. StableToolBench benefits the most from coordination, but it also sits furthest to the right on cost.

weakly supported hypotheses rather than fixing them. On plancraft, the dominant problem is premature termination, so additional discussion is only useful if it forces the system to keep alternate routes alive. On stabletoolbench, discussion can genuinely help when multiple agents assemble partial evidence from tools, which explains why group_chat_debate wins despite the large cost increase. On workbench, however, coordination cannot rescue missing entity bindings unless one of the agents actually resolves the missing contact or sender information. For finance_agent, the evidence suggests that additional orchestration mostly changes how the system expresses uncertainty rather than eliminating the underlying support gap.

The cost side of the trade-off is real. The best stabletoolbench topology uses 9.7× the tokens of sas; the best finance_agent topology uses 8.4×. This does not mean the gains are meaningless. It does mean the paper should not frame topology choice as a pure quality decision. Coordination changes both outcome and budget.

9.1. When Coordination Helps

The clearest positive case for coordination is stabletoolbench. The benchmark requires partial evidence from multiple tools, and the winning topology is discussion-heavy. This is the benchmark where multi-agent coordination appears to add genuine value rather than merely additional tokens. The likely mechanism is not abstract debate for its own sake; it is the ability to assemble partial, heterogeneous evidence into a more complete local state before answering.

9.2. When Coordination Mostly Amplifies Failure

The weakest case for additional coordination is browsecomp. Here the main difficulty is not lack of conversational diversity but lack of decisive evidence. Once the system has locked onto a weakly supported candidate, discussion often expands that weak hypothesis rather than replacing it. Finance_agent shows a related but distinct pattern: richer orchestration changes the amount of reasoning around the evidence more than it changes the quality of the underlying support.

9.3. When Coordination Is Orthogonal to the Bottleneck

In workbench, the main obstacle is often a missing person, sender, or assignee binding. That means the utility of extra coordination is structurally limited unless one of the agents can actually resolve the missing entity. Plancraft is similar in a different way: the bottleneck is often a local impossibility judgment. Additional coordination helps only if it actively keeps alternative routes alive; otherwise it just increases the conversational path to the same early stop.

9.4. The Coordination Tax

The token multiplier in Figure 3 can be read as a coordination tax: the extra deliberation cost required to purchase one unit of quality improvement over the single-agent baseline. On stabletoolbench, the tax appears high but at least partly justified, because the benchmark shows the largest success lift and also demands cross-tool evidence assembly. On plancraft, by contrast, the marginal utility of additional discussion is poor. The best system improves on sas by only 0.011 success points while spending 3.9× the tokens. This is precisely the kind of profile that suggests expensive redundancy rather than genuine collaborative gain.

The notion of coordination tax is useful because it changes how we interpret multi-agent wins. A topology should not be credited merely for spending more context to recover a few additional cases. We should ask what kind of deliberation the extra budget is buying. In several failure traces, the agents appear to repeat, paraphrase, or politely ratify the same intermediate claim rather than subjecting it to adversarial verification. That pattern is close to sycophantic coordination: the group sounds collaborative, but the informational diversity is low. From a deployment perspective, this matters because production systems pay for every extra round. If the added communication does not significantly alter the evidence state or improve ad-

versarial robustness, then the collaboration tax may be economically and operationally unjustified.

10. Reproducible Triggers

The traces also expose compact trigger shapes that recur often enough to be useful to other researchers:

- repeated query reformulation with little change in retrieved evidence,
- immediate impossible after the first observed local shortage,
- multiple tool retries after empty or inconsistent external outputs,
- blocked workflow steps because a person, sender, or assignee remains unresolved,
- detailed final answers whose support chain is thinner than their confidence level suggests.

These triggers are valuable because they can be recognized before a full benchmark score is computed. They are not universal causal laws, but they are stable enough to serve as debugging handles. A reviewer or system builder can inspect a trace and ask whether the run is learning new evidence or merely expanding text around a fixed uncertainty.

10.1. How Errors Compound

One useful way to interpret these triggers is through a state-space view of execution. Each tool call, retrieval step, or entity binding updates the agent's current state, and later decisions are only as reliable as the latent state they inherit. Under this view, many failures in our audit are not isolated bad actions but examples of local fault becoming global failure. A missing entity binding in workbench corrupts the action space available to later workflow steps. A weak retrieval branch in browsecomp narrows the hypothesis space too early and causes later deliberation to condition on a distorted evidence state. A sparse tool response in stabletoolbench can propagate upward into unsupported synthesis even when no single later message is individually unreasonable.

This perspective helps connect our qualitative audit to ideas from causal inference and formal verification. If the run is treated as a state machine, then a useful intervention target is not just the final answer policy but the transition rule between states. The right question becomes: which upstream state transition injected the error that later became irreversible? In some benchmarks the answer is a retrieval update; in others it is an unjustified impossibility transition or a failed grounding event. Framing the problem this way clarifies why local correctness is insufficient for logical stability. A run can contain many locally sensible steps while still drifting into a globally invalid state because the wrong latent assumption was never repaired.

11. Diagnostic Extensions and Repair Opportunities

The audit suggests several concrete repairs.

- Search-stagnation detectors should monitor whether retrieval is changing the evidence state rather than merely producing more snippets.
- Impossibility standards should distinguish local dead ends from globally impossible states.
- Tool fallback policies should stop long retries when the interface has already signaled a low-information regime.
- Early grounding passes should resolve people, emails, senders, and assignees before deeper workflow planning begins.
- Support-sensitive answer policies should force financial and factual tasks to report uncertainty when the evidence chain is thin, even if a plausible answer has been generated.

One natural next step is a counterfactual replay study: replay the same local subtask without inter-agent communication and compare the outcome and token cost. We deliberately describe this as a diagnostic extension rather than as a completed result. The current paper motivates such a replay by showing that many expensive failures are trace-visible before they are benchmark-visible.

We also see two promising methodological intervention directions. First, future work could use causal inference over trace events to isolate how much a single tool failure contributes to the final unsupported answer. This would make it possible to separate hallucination caused by weak external evidence from hallucination caused by downstream synthesis behavior. Second, benchmarks such as plancraft and workbench suggest the value of formal verification over the current evidence state. Before an agent is allowed to emit impossible or proceed with a high-impact workflow action, the system could be required to satisfy explicit logical constraints over what has and has not been verified.

Observed trigger	Benchmark context	Downstream failure mode	Diagnostic metric
Repeated query reformulation with overlapping snippets	Browse-style multi-hop retrieval	Evidence insufficiency or long-horizon drift into a wrong hypothesis	Low unique-doc growth across search turns; low evidence-state change between retrieval steps
Immediate impossible after the first observed local shortage	PlanCraft inventory and recipe planning	Premature impossibility	Count of alternate route expansions before termination; first impossibility issued before search frontier is exhausted
Multiple tool retries after empty, sparse, or inconsistent outputs	Tool-mediated API tasks in StableToolBench	Tool/interface fragility	Tool error rate, repeated identical API arguments, low-information response streak length
Workflow stalls after unresolved person, sender, or assignee lookup	WorkBench calendar / CRM / directory workflows	Entity grounding failure or workflow blocking	Missing-entity lookup count; unresolved binding before first actionable tool update
High-confidence numerical answer with thin or conflicting evidence chain	Finance-agent quantitative synthesis	Financial support gap or stack brittleness	Citation / support density, evidence-source disagreement count, answer specificity exceeding support coverage

Table 5. Trigger-to-failure mapping. The goal is to move from qualitative trace reading toward programmatic diagnostics that future systems can assert over their own execution traces.

12. Discussion

The broader implication of this study is methodological. Agent evaluation should not stop at terminal success tables, especially in long-horizon or tool-mediated settings. When a benchmark release includes only scores, it becomes difficult to distinguish between brittle retrieval, unsupported synthesis, premature termination, operational grounding failure, and long-horizon drift. These error classes imply different repairs, different safety concerns, and different expectations about transfer.

The paper also suggests that benchmark designers should consider traces as part of the benchmark interface. A benchmark that preserves prompt structure, tool calls, stage contracts, and final evaluation outcomes is far more useful for diagnosis than one that records only whether the final answer was correct. This does not mean every benchmark must mandate a particular trace schema, but it does mean that trace completeness has direct scientific value. In particular, operational definitions with explicit trace boundaries make it easier to compare studies and to test whether a claimed failure mode is genuinely reproducible.

Finally, the results speak to a common mistake in multi-agent evaluation: treating additional coordination as a free source of capability. The best systems in this suite are benchmark-dependent, and the cost-quality trade-off varies sharply across environments. A realistic evaluation agenda therefore needs both process visibility and cost visibility. Without the first, we do not know what failed; without the second, we do not know what the improvement actually cost. This is also why we frame our labels as falsifiable mechanistic hypotheses rather than as purely descriptive categories: the point is not only to name failure, but to make it testable.

13. Threats to Validity

The main threat to validity is scope. The paper analyzes one experiment suite and one model family, so the audit should be read as a structured diagnosis of this corpus rather than a universal law of agentic failure. A second threat is labeling subjectivity. The audit uses a single primary label per run and combines heuristic routing with human trace inspection; some borderline cases could reasonably support a different label or a multi-label interpretation.

Another threat is benchmark dependence. Categories such as premature impossibility are much easier to identify in plancraft because the benchmark action grammar is explicit, whereas categories in browse-style or finance-style tasks require more interpretive judgment. Finally, some apparent differences between topologies may reflect benchmark-specific interactions with prompts, tool affordances, or evaluation scripts rather than deeper truths about coordination itself.

14. Limitations

This paper has several limitations. First, it analyzes one experiment batch and one model family. The taxonomy should therefore be read as a structured observation over this corpus, not as a universal theory of agent failure. Second, the failed-run audit uses one primary label per run, even though some runs clearly exhibit mixed failure. Third, the audit protocol combines phrase-level cues with representative trajectory inspection rather than a large formal annotation study with multiple raters. Finally, because the paper focuses on diagnosis, it does not yet include a validated intervention study showing that the proposed repairs improve the audited failure modes.

15. Conclusion

Agentic failures are not well described by terminal accuracy alone. In a cross-benchmark MAS Analyzer suite, the best topology depends on the benchmark, and the dominant failure mode depends on the task family. A failed-run audit over 330 failures shows recurring benchmark-specific regimes: evidence insufficiency in browse-style search, premature impossibility in planning, tool/interface fragility in tool-heavy tasks, grounding and workflow blocks in workplace tasks, and support fragility in financial QA. These findings argue for a simple shift in evaluation practice: traces and trajectories should be treated as core outputs of agent benchmarking rather than as optional debugging artifacts.

Impact Statement

This paper studies failure in agentic AI systems with the goal of improving reliability, transparency, and reproducibility. The immediate positive impact is methodological: better trace diagnostics can help researchers distinguish unsupported answers from well-supported ones, identify brittle tool interfaces, and design more targeted repairs. Potential negative impacts come from dual use. A detailed understanding of failure triggers could be used to intentionally induce agent breakdowns or to optimize systems toward benchmark-specific artifact exploitation.

The societal risk is not limited to obvious crashes. In finance_agent, the dominant failure mode is often a fluent and numerically specific answer with a weak support chain. If such behavior were deployed in an automated financial reporting or trading pipeline, it could create more serious governance and market risk than an explicit refusal, because the system appears confident while failing a faithfulness test. We therefore recommend that failure taxonomies be reported together with explicit uncertainty, benchmark scope, and the limitations of each diagnosis.

References

Liu, X. et al. Agentbench: Evaluating LLMs as agents. arXiv preprint arXiv:2308.03688, 2023. URL https://arxiv.org/abs/2308.03688.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.

Shinn, N., Cassano, F., Berman, A., and Gopinath, A. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, 2023. URL https://arxiv.org/abs/2303.11366.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Wang, C., et al. Autogen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023. URL https://arxiv.org/abs/2308.08155.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023. URL https://arxiv.org/abs/2210.03629.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., et al. Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854, 2023. URL https://arxiv.org/abs/2307.13854.

A. Audit Scope and Reproducibility

The audit reported in the main paper uses all failed runs under the best-performing topology for each benchmark. This design keeps the audit tractable while ensuring that the failures are not artifacts of a deliberately weak topology. The artifact bundle contains benchmark-level summaries, topology-level descriptors, per-task trajectories, event-level traces, and evaluation JSON files. A reviewer can reproduce the audit logic by re-selecting the best topology per benchmark from the summary table, enumerating the failed runs via the evaluation outputs, and then re-reading the trajectories in execution order.

B. Coding Rubric

The audit uses the following primary labels:

- Evidence insufficiency / unsupported hypothesis: the run returns a candidate answer without a support chain strong enough for the benchmark requirement.
- Explicit evidence starvation: the run directly reports that evidence could not be recovered or that retrieval remained blocked.
- Premature impossibility: the run terminates with impossible after a local shortage or local dead end rather than an exhausted global search.
- Tool/interface fragility: the run is blocked primarily by empty, inconsistent, or insufficient tool outputs.
- Unsupported synthesis: tools return some evidence, but the final answer exceeds what that evidence supports.
- Workflow blocking: the run stalls on an operational precondition without a clean entity-grounding diagnosis.
- Entity grounding failure: the run fails because a person, sender, email, or assignee cannot be resolved.
- Financial support gap: the run returns a detailed financial answer with a weak or incomplete support chain.
- Stack brittleness: the run fails due to interacting evidence, tool, and benchmark-interface problems that are not well captured by a single narrower label.

C. Audit Decision Rule

When multiple labels are plausible, the audit chooses the earliest trace-visible condition that best explains why the run could not satisfy the benchmark objective. This rule intentionally favors diagnostic usefulness over maximal label granularity. For example, if a tool returns sparse data and the final answer is unsupported, the run is labeled as tool fragility when the sparse tool output is the more informative explanation of the failure path.

D. Additional Case Notes

D.1. BrowseComp

Representative failures often end with either a blocked answer or a concrete but weakly supported hypothesis. In one task, the run repeatedly narrowed around a candidate institution before the supporting chain had become complete. In another, the final answer explicitly stated that the necessary evidence had not been recovered from the available corpus.

D.2. Plancraft

Representative failures are short and diagnostic. A common pattern is a final answer of the form “impossible” followed by a local shortage, such as missing dye ingredients or a missing smelting path. The trace rarely contains a persuasive attempt to exhaust alternate routes before termination.

D.3. StableToolBench

Representative failures often cite weak or inconsistent APIs. Typical patterns include sparse medical guidance retrieval, inconsistent geographic data from property tools, and synthesis attempts after the tool evidence has already stalled. These traces are especially useful for separating tool failure from downstream reasoning failure.

D.4. Workbench

Representative failures frequently mention missing emails, unresolved assignees, or blocked sender identification. The trace often preserves a perfectly reasonable workflow plan up to the point where execution requires a concrete identity binding.

D.5. Finance Agent

Representative failures are more subtle. Some produce highly specific numerical answers with weak support chains; others explicitly struggle with filings, metric definitions, or conflicting evidence sources. This is why the benchmark benefits from a separate label for support fragility rather than being collapsed into generic retrieval failure.

E. Trace Schema

The artifact schema includes the following fields when available:

- benchmark name and task identifier,
- topology and model name,
- run index,
- final answer, final reason, and evaluation outcome,
- event-level tool calls and tool errors,
- agent messages and communication structure,
- token usage, latency, handoff count, and other trace metrics.

For qualitative review, we read each failed run in execution order: task prompt, stage contracts, visible packets, tool outputs, final answer, and evaluation outcome. This preserves temporal structure and makes it easier to identify the point at which a run transitions from productive evidence gathering into self-reinforcing uncertainty or premature termination.

F. Additional Quantitative Notes

Within the best-topology slice, the failed-run distributions are informative in their own right. Browsecomp has only 10 successful runs out of 90, which means most opportunities for improvement lie in better evidence control rather than in small ranking refinements. Plancraft succeeds more often overall, but when it fails, it fails in a highly concentrated way. Stabletoolbench shows the strongest trade-off between quality gain and token cost, which is consistent with a benchmark where coordination is genuinely useful but expensive. Workbench and finance-agent both show low absolute success despite very different trace shapes, underlining the value of benchmark-specific diagnosis.